

ENHANCING HUMANOID ROBOT FUNCTIONALITY THROUGH VISION-BASED NAVIGATION WITH FALL RECOVERY AND OBJECT MANIPULATION

Bal Krishna Shah^{*1}, Sabin Acharya¹, Safal Karki¹, Aayush Pathak¹, and Saroj Shakya¹

¹Department of Electronics and Computer Engineering, Thapathali Campus, Institute of Engineering, Tribhuvan University, Kathmandu, Nepal

*Corresponding Email: bal.077bei010@tcioe.edu.np

DOI:

Abstract:

The robotics sector struggles to integrate vision-based navigation on a bipedal humanoid robot capable of performing human-like tasks. Although the use of ultrasonic sensors and infrared sensors is a traditional method for object detection, it has significant drawbacks such as low range, high cost and sensitivity to the environment. "Enhancing Humanoid Robot Functionality Through Vision-Based Navigation with Fall Recovery and Object Manipulation" proposes to give vision to the robot, making it capable of transporting objects from one location to another. The two ESP32-CAMs are used as a stereo camera for image capturing, employing the use of YOLOv11 for object detection, and the principle of the stereo camera for depth calculation. With the use of one of the most robust and accurate object detection algorithms available, the project aims to enhance object transportation within the visual range of the robot. The final robot can navigate intelligently and grab objects using image processing. The developed humanoid robot encompasses the feature of automatic fall recovery in simulation and natural human movement patterns through kinematical calculations, showcasing potential applications in hazardous environments, industrial automation and interplanetary exploration.

Keywords: Autonomous Navigation, Degree of Freedom, Distance Estimation, Humanoid Robot, YOLOv11

1. Introduction

The concept of a humanoid robot has been around for quite a long time and has long captured the imagination of researchers and the public. Humanoid robots are designed to replicate human form and functionalities, with human-like structures that incorporate two arms, two legs, a torso, and a head. With evolving automation systems, the relevance of the integration of human-like machines in the industrial and service sectors has increased, along with expectations of improved task efficiency, accuracy, and autonomy [1]. The popularity of humanoid robots, increased by Sci-Fi movies, has further fueled interest and innovation in this field.

Practical developments of humanoid robots, such as Hanson Robotics' Sophia, capable of facial expressions and conversation, and Boston Dynamics' bipedal systems capable of dynamic movements, including jumping and rolling, show the real-world potential of humanoid robots [2, 3]. Similarly, events like Robo-

One in Japan demonstrate innovation and agility in bipedal robots through competitions, encouraging the practical and entertainment value of these systems.

A typical humanoid robot is designed to mimic the human structure by implementing major joints such as the neck, shoulder, elbow, hand, pelvis, knee, and ankle, each requiring its own motor and range of motion [4].

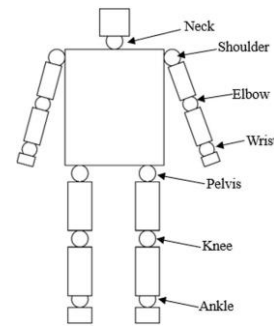


Fig. 1. Major Joints in a Human Body

A key advantage of humanoid robots lies in their structure, which enables them to operate in human-centric environments without the need to redesign existing tools and infrastructures [1]. As AI, machine learning, and hardware continue to advance, the use of techniques such as reinforcement learning and imitation learning allows robots to learn and replicate complex human behaviors from data [5, 6].

However, significant challenges remain in developing humanoid robots, particularly in the domain of vision-based navigation. Unlike wheeled robots, bipedal robots need to maintain balance while navigating uneven and dynamic environments [7]. Real-time decision making, visual perception and object manipulation require accurate sensors and high-performance computing [8], which are often expensive and difficult to integrate efficiently and effectively within the constraints of a humanoid platform.

This work aims to develop a cost-effective humanoid robot addressing the challenge of integrating vision-based navigation for object recognition using a stereo-camera setup. The robot is designed to recognize objects, estimate distance using stereo vision, and perform autonomous navigation, pick and place task utilizing YOLOv11 object detection model [9], and geometric methods for distance estimation, allowing it to navigate visually to the object's location [7, 10].

The next section gives a concise review of the lit-

erature in related subjects and the work done. Section 3 gives an overview of the system and describes each major component of the system. Section 4 presents detailed information on the methods and materials used. Section 5 provides implementation details. Section 6 discusses the results and performance analysis, followed by the conclusion and future work.

2. Related Work

An integrated framework is proposed for mobile robots operating in unfamiliar environments based on YOLO-GGCNN, which combines accurate map construction using a multi-robot cooperative simultaneous localization and mapping (SLAM)-based algorithm with a two-stage grasping module [11]. The grasping module that uses YOLOv4 for object detection and YOLO-GGCNN (You Only Look Once-Generative Grasping CNN) for grasp predictions transmitted to the robotic arm through a Robot Operating System (ROS) achieved a captured accuracy of 86.0% with a detection time of 0.11 seconds per frame. The researchers validated the framework by testing it in various environments and making comparisons with existing approaches.

An Object Detection and Recognition algorithm for a pick and place was developed in which the system extracts features from the cluttered scenes, localizes and preprocesses the object and uses a classifier to output object type and coordinates for robotic manipulation [12]. The research shows high accuracy in image processing and integration potential in a physical robot.

Moreover, a stereo camera setup was developed to measure the distance between the camera and human face by calculating the shifts in pixels between the faces from each cameras through the use of disparity maps [13]. Despite requiring calibration for each measurement, the accuracy was found to be decent for small distances, with a diminishing result for increased distance.

Another research developed a vision-based obstacle detection and path-finding algorithm for NAO humanoid robot using Support Vector Machine (SVM) and A* algorithms [14]. The algorithms are used for classification of the captured images and finding an optimal path between the start and the end point [15].

In relation to the kinematics of the bipedal robot, an inverse kinematic solution is proposed for an 11-DOF bipedal robot using geometric methods [16]. The bipedal robot includes five links joined by revolute joints that include the ankle, knee, and hip joints. The research makes the Inverse Kinematics (IK) problem easier to understand by symbolically reducing the complexity of solving univariate polynomials and produces an analytical solution for a given end-effector position. A robot model using lightweight modular components based on human joint ratios is presented, and a kinematic model is formulated using Denavit-Hartenberg parameters (DH) to derive transformation matrices for each joint. This paper provides a methodical approach to solving the IK problems for bipedal

robots.

A comprehensive IK model for an 18-degree-of-freedom robot was investigated specifically on the Bi-oloid Premium Robot platform [17]. The study addresses the shortcomings of earlier works, incorporating detailed kinematic models for both upper and lower body segments. It presents a systematic methodology for modeling the robot's forward and inverse kinematics using the DH method, with each joint characterized by specific transformation matrices. The robot is segmented into four open kinematics chains, starting from the support feet to the hands and pelvis. This approach allowed precise control of a high-DOF biped system.

In another work related to the design of a humanoid robot, a bipedal robot is proposed based on the kinematics and dynamics of human's lower limb [18]. The robot's design primarily focuses on lower limbs with a high-torque hip joint combined with 2-DOF parallel mechanisms and a high-torque knee joint with variable damping. The robot uses a distributed control system, dividing into a main control module, a motion execution model and a sensor module, along with other integrations of IMUs (Inertial Measurement Units) and pressure sensors. This work serves as a base for the distributed control of a mobile robot.

A Vision-Based following system for mobile robots was implemented in which a mobile robot tracks and follows a tele-operated master robot using a PID (controller [19]. The follower robot employing a Kinect sensor follows the master robot controlled manually. The follower robot identifies the x, y and z position using the camera and the depth data from the sensor to find the angle and distance between the master and the follower robot, which is given as the error of a PID controller.

A walking pattern generation and trajectory planning method for humanoid robots is presented, focusing on the planning of the walking cycle of a humanoid robot with a total of 5 joints in each leg in 9 steps [20]. This work describes two methods of holding balance in humanoid robots. One by calculating the COG (Centre of Gravity) of the robot, in which for the robot to be balanced, the COG must lie within the support polygon of the robot. Another method is to find the ZMP (Zero Moment Point). Among these, the described work utilizes the ZMP method using FSR (Force Sensing Sensor) values attached to the sole of each foot.

An important research about rapid motor adaptation (RMA) in dynamic legged robots to successfully walk on different terrains [21], an algorithm for reinforcement learning with a bioenergetics-based reward system used to train a quadruped robot in a simulated environment, then tested in controlled physical terrain for performance assessment. The model includes terrain and dynamics encoding for online adaptation. The model tested on Unitree's A1 robot, effectively adjusted to varying surfaces and dynamic payloads.

While previous related works have addressed various aspects of designing humanoid robots, kinemat-

ics and object interactions, providing great insights on the challenges, methods and limitations of developing an integrated system for different humanoid platforms, most of them rely on simulation and sophisticated hardware, limiting real-world accessibility [1,7]. In contrast, the system and design presented here focus on developing a low-cost, modular system using off-the-shelf components, aiming to bridge the gap between theoretical research and practical, affordable implementation.

3. System Overview

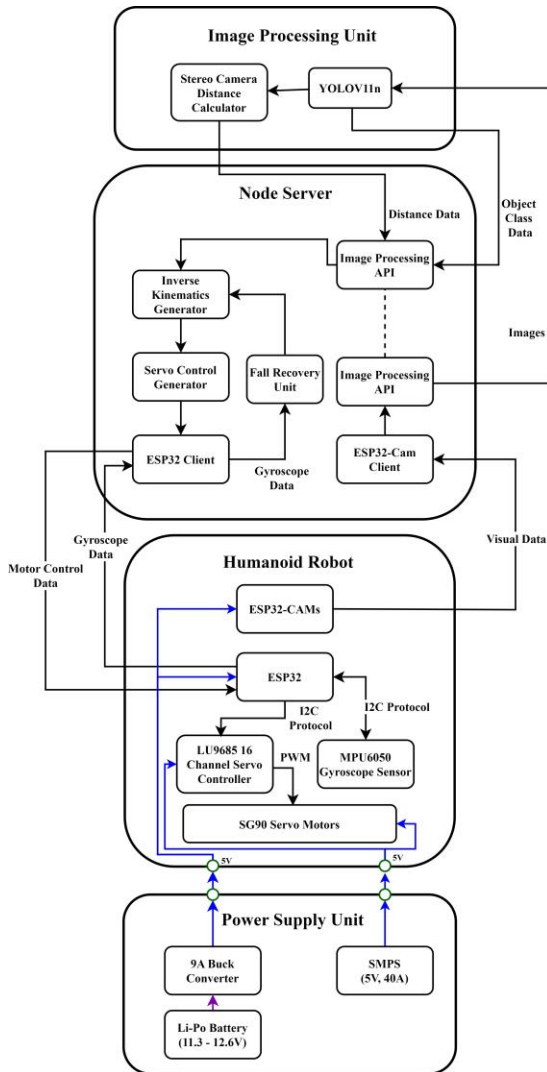


Fig. 2. Block Diagram of the System

3.1. Humanoid Robot

The Humanoid Robot is a robotic system made up of a LU9685 16-Channel Servo Controller to control the servo motors, MG996R servo motors to move the robot's joints, and ESP32-CAMs to record images. Additionally, an ESP32 micro-controller uses the generated commands from the computer's server to control the robot's arms and legs. Additionally, an MPU6050

gyroscope sensor is included to identify the equilibrium state. The Humanoid Robot's body is 3D printed.

3.2. Node Server

WebSockets are used by the Node server to send and receive data from other units. The servo control generator, which has the data required for the ESP32 to drive the servo motors, receives the data from the Image Processing Unit. Additionally, the Node Server sends the information from the robot's ESP32-CAM images to the Image Processing Unit.

3.3. Image processing Unit

A distance calculator that determines the separation between the item and the robot is part of the image processing unit. This unit determines the distance by using the YOLOv11 to recognize objects in the stream that was received from the stereo camera configuration.

3.4. Power Supply Unit

The power supply unit drives the power into the whole humanoid robot. It consists of a 3S Li-Po battery, a 9A Buck Converter and a 5V 40A Switch Mode Power Supply.

4. Materials and Method

4.1. Robot Design and Kinematics

The robot comprises 19 DOFs, each consisting of a servo motor to represent joints. The division of DOFs in different parts of the robot's body is as follows:

- Head: 1 DOFs for Neck Joint
- Torso: 0 DOFs (Circuits and Controllers)
- Arms: 3 DOFs in each arm for a total of 6 DOFs with 2-Shoulder, 1-Elbow
- Legs: 6 DOFs in each leg for a total of 12 DOFs with 3-Pelvis, 1-Knee, 2-Ankle

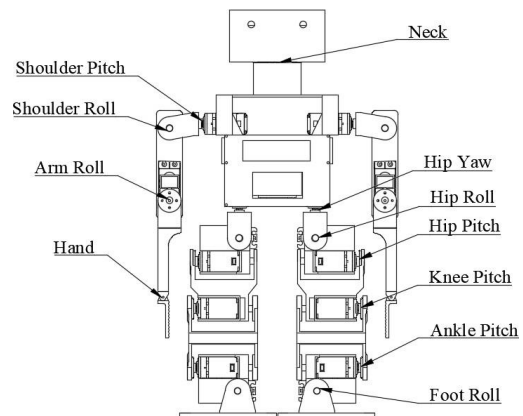


Fig. 3. Robot's Model with all DOFs

The combined action of 6 motors in each leg allows 3 different kinds of motion of the robot, namely: forward motion, sideways motion and rotation motion. Similarly, the combined action of 3 motors of each hand allows the pickup motion of robot and also in balancing the robot while walking. The degree of freedom

in the head allows the robot to rotate the head for detecting the object via the stereo camera setup used in the head for object detection and distance estimation. Forward and Inverse kinematics solved by the simulation software, with modifications to comply with the real environmental characteristics, are used for controlling all the motions of the robot. Motion data is divided into angle frames, and then the in-between frame values are linearly interpolated during the motion execution to obtain smooth motion.

The basic kinematic structure of joints and links is shown in fig.4. Although most simulation software readily have tools for solving kinematics problems, the Denavit-Hartenberg method forms the basis for the calculation of Forward Kinematics to obtain the end-effector position (Cartesian coordinates) for given joint angles. It also provides the framework for formulating equations of Inverse Kinematics for more dynamic motion systems of robots.

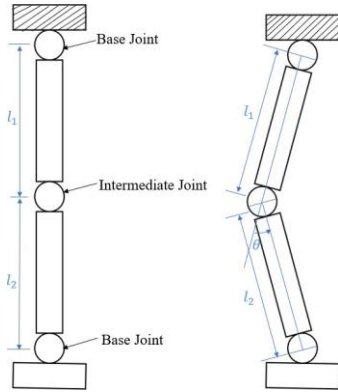


Fig. 4. Basic Kinematic structure of the robot

Based on the orientation and position of the joints, the Denavit-Hartenberg (D-H) parameters are derived and used to formulate a homogeneous transformation matrix as shown in Equation 1:

$$T_{n-1}^n = \begin{bmatrix} \cos \theta_n & -\sin \theta_n \cos \alpha_n & \sin \theta_n \sin \alpha_n & r_n \cos \theta_n \\ \sin \theta_n & \cos \theta_n \cos \alpha_n & -\cos \theta_n \sin \alpha_n & r_n \sin \theta_n \\ 0 & \sin \alpha_n & \cos \alpha_n & d_n \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

Then, for each joint shown in Fig. 4, a transformation matrix T_{i-1}^i is computed using the respective joint angle θ_i , link offset d_i , link length r_i and twist angle α_i [22]. The overall transformation matrix from the base frame to the end effector is obtained by the sequential multiplication of individual transformations:

$$T_4 = T_1 \cdot T_2 \cdot T_3 \cdot T_4 \quad (2)$$

Each leg forms its kinematic structure with respective D-H parameters, with one leg being the mirror of another. An example of D-H parameters of a leg is shown in table.1. A similar configuration is followed for both legs, and the required calculations are done to solve the kinematics problems of different legs and arms based on the requirement. Then the joint angles are mapped to the servo motors on each joint of

the humanoid robot to make the robot walk, pick objects and perform other motions. While doing so, a balanced gait is maintained by employing different methods such as lowering the COG (Center of Gravity) and increasing the foot base area.

Tab. 1. DH-Parameters Configuration for Legs

Joint Pair	d [m]	θ [deg]	r [m]	α [deg]
Hip_Yaw - Hip	d_1	θ_1	0	90
Hip - Thigh	d_2	θ_2	a_2	-90
Thigh - Knee	0	θ_3	a_3	0
Knee - Ankle	0	θ_4	a_4	0
Ankle - Feet	d_5	θ_5	a_5	90
Feet - FootTip	d_6	θ_6	a_6	90

4.2. Electronic Architecture

The electronic and electromechanical components used for the robot's operation are controlled by an ESP32-Wroom32 micro-controller. Due to a lack of 19 active PWM pins to control 19 servo motors, an additional 16-channel servo controller, LU9685-20CU, is used, which uses a micro-controller from the STC8H family. The module is controlled via ESP32 using the I2C communication protocol. The daisy chaining of the MPU6050 sensor with the LU9685-20CU servo controller is done to receive the accelerometer and gyroscope data from the MPU6050 sensor. The overall electronics architecture and the circuit diagram is shown in fig.??.

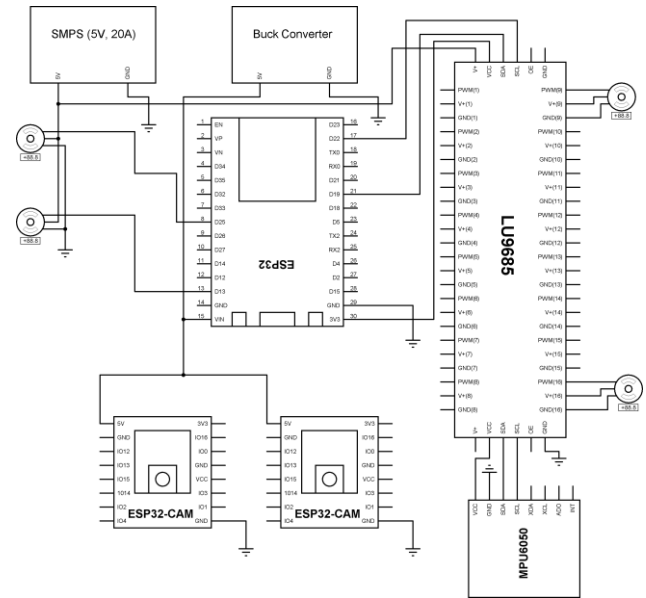


Fig. 5. Circuit diagram of the electronic system of the Robot

For powering the motors, a 5V, 20A SMPS is used. Due to the transient drop in voltage during current spikes, the SMPS could not be used as a reliable source to power up the micro-controllers, which reset as the voltage dropped. So, an additional 9A buck converter setup at 5V and powered by a 3-cell Li-Po battery is

used to power up the ESP32 and ESP32-CAMS. The ESP32-CAMS are isolated from the other circuit components as it sends the image data directly to the server via a Wi-Fi network. The ESP32 also receives the control data to move motors via a remote server through a Wi-Fi network.

4.3. Object Detection and Distance Estimation

The YOLOV11n state-of-the-art object detection model trained on a custom dataset of the cube to be detected is used for object detection. The model runs on the server using the ONNX (Open Neural Network Exchange) runtime, which allows running the machine learning models in the Node.js server too. The bounding boxes are estimated for the object in both cameras.

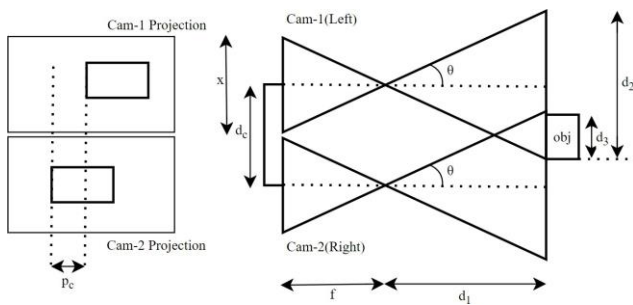


Fig. 6. Stereo Camera Setup for Distance Estimation

The pixel offset between the images detected from the camera is used to calculate the distance between the object and the camera using the formula:

$$d = f + \frac{Nd_c}{2p_c \tan \theta} \quad (3)$$

Where d is distance to be estimated, f is the focal length of the camera, p_c is the pixel offset between the images captured from two cameras, N is the total pixel width of the image, and d_c is distance of separation between the cameras.

The value of $\tan \theta$ and focal length needs to be calibrated, for which two known distances d_1 and d_2 with their corresponding pixel offsets p_1 and p_2 are used and calculated using the equations 4 and 5.

$$\tan \theta = \frac{Nd_c}{2(d_1 - d_2)} \left(\frac{1}{p_1} - \frac{1}{p_2} \right) \quad (4)$$

$$f = d - \frac{Nd_c}{2p_1 \tan \theta} \quad (5)$$

4.4. Control Logic and Fall Recovery

The control system for the humanoid robot consists of a combination of both closed-loop and open-loop control systems. The robot requires a closed-loop control system as the motions are not perfect and consistent. During the motion, the initial position of the robot plays a very crucial role in how the robot runs the whole motion. At the end of the motion, ideally the robot should return to the standing initial position, but

due to the mass shifts during the motion and the small wobbling issue of each motor in its each rotational position, the robot's position differs a bit from the actual initial position at the end of each motion (however, the code is written to bring back to its initial position).

As a result, the robot's walk and other motions are not always completely the same. And the motion is also dependent on the walking surface friction. So, the distance covered by the robot in motion cannot be the same all the time. This induces the need for a closed-loop control system where the robot, after each/multiple motions, takes feedback from the camera and the distance estimation unit to locate the object that it is intended to pick. The robot requires an open-loop control system due to some of the physical limitations of the robot. The robot has only one degree of freedom in its head that allows it to rotate in the z-axis only so that the robot can rotate its head and see the objects around it. The robot has no degree of freedom to bend its head up/down. As a result, when the robot approaches the object, the view of the object becomes smaller and smaller for the robot, and at a certain distance, the object is completely below the view range of the robot. In that case, the robot has no idea where the object is, and the control system fails. So, an open-loop control system is setup so that after a certain distance, the robot takes one cycle of forward walking and then picks up the object without exact calculation. This open-loop control system is accurate if the motions of the robot are stable and consistent enough.

For fall recovery, the MPU6050 sensor data is used, which provides the accelerometer and gyroscope data readings continuously. The pitch and roll angle can thus be calculated from that data using the equations 6 and 7

$$\theta = \tan^{-1} \frac{A_y}{A_x + A_z} \quad (6)$$

$$\phi = \tan^{-1} \frac{A_x}{A_y + A_z} \quad (7)$$

Where θ is Pitch Angle, ϕ is Roll Angle, A_y is Acceleration along Y-axis, A_x is Acceleration along X-axis, and A_z is Acceleration along Z-axis.

A threshold is set for the robot to be considered in the fallen state. If the robot's pitch or roll angle is found to remain within that threshold for a significant time, the current motion of the robot is halted, and the fall recovery motion is initiated. However, due to bulky design and fragile motors, the fall recovery could not be implemented in physical robot but is implemented in simulation. In the physical robot, only the fall is detected and notified in the mobile application.

4.5. Simulation Setup

To simulate the motion dynamics of the robot, the designed model of the robot was first exported from the design software in .OBJ format as an optimized mesh format combining insignificant parts to reduce the model complexity for simulation. The model was

then imported to CoppeliaSim (a robotics simulation software) with a custom scaling factor to match the real-world scale of the robot. The imported model was Convex decomposed using CoppeliaSim's built-in tool to convert complex shapes with high triangle count into primitive convex shapes to enhance simulation stability and efficiency.

Subsequently, the joints were manually defined and positioned according to the kinematic structure of the robot with each joint set to position control to emulate the servo motor behavior. The physical properties of the model, such as: mass of each component and torque applied at each joint, was also adjusted to reflect the real-world parameters and dynamics of the real-world. The simulation software provides Lua/Python scripting support for programming the behavior of joints. The PyBullet engine was used for rendering physics.

5. Implementation

5.1. Simulation Implementation

The mass of every component and the joint torque must match the actual values in order to mimic the robot using real-world parameters. This was accomplished by altering each component's dynamic attributes, from the body to the links. The total mass of the body was adjusted to roughly 2 kg in the simulation by giving accurate mass to every part of the robot, including the motor and battery weights. Likewise, 6 Nm was the torque applied to the joints.

Through the use of Lua scripts, the joints were programmed in accordance with the various sets of motion. The joint angles were programmed to change frame by frame. Each frame had its own data of joint angles generated to produce a seamless motion. Each type of movement, depending on its complexity, required a varying number of frames. The robot model was programmed to change its state after each frame and decide the next state according to the control logic.

5.2. Motor Initialization

This is the main stage in which the motor keeps its angle to keep the body of the robot upright. For their positions to remain stable, all of the motors must be adjusted to their angle with respect to the LU9685. Without the motor initialization, the circuitry would not supply power to the robot's motor, which would be unable to support the body because of the strong torque exerted by the body. As a result, the robot collapses. This is the first thing the robot does when it is first initialized. The Humanoid Robot program is used to accomplish this by first setting the starting frame angle to each motor joint individually.

5.3. Dynamic Environment Implementation of Data from Walking Simulation

Because the simulation data is implemented dynamically, the robot is free to engage with its environment and carry out the necessary action (in this case, walking simulation). The robot is impacted by gravity, friction, and other real-world physical factors in

this setting. In a physical implementation, the robot is placed on the ground or another surface it must walk on, but in a simulation, this is accomplished simply by maintaining a dynamic environment. With the same mass and joint location as in the actual world, the robot was stable in the simulation; however, because each frame had such a short time allotment, the joint angles moved more quickly, which resulted in a sharp increase in momentum.

To solve that issue, the time allocated for each frame to complete was increased fourfold. As a result, the problem of overthrow was solved.

The majority of the frames were unstable and led to the robot falling because of imbalance, while other frames of motion were directly stable when applied to the robot's body. Upon closer examination, it was discovered that the robot's movements to get to the frame position were what were causing the imbalance, rather than the frame position itself. This is because the robot splits the time allotted for those frames into 50-ms time slots, determines the incremental angle that has to be taken for each joint in each 50-ms time slot, and moves each joint nearly simultaneously. Because of this, there were unanticipated intermediate frames where the robot's equilibrium failed, and it fell.

Intermediary frames were introduced to balance the robot between the simulation frames where it lost control in order to address this problem. This stage was done by hand, using the motion planning page of the Humanoid Robot application to manually operate each motor in order to obtain intermediate frames. The robot was more evenly balanced with the inclusion of intermediate frames.

5.4. Physical Implementation of Pick-Drop Movement

The robot fell right from the first frame since it was not feasible to integrate the pick-and-drop movement simulation data directly in a real-world setting. After that, the robot's bending angle had to be set so that it wouldn't fall. The hands' pickup motion then resembled the simulation data nearly exactly.

5.5. Physical Implementation of Rotation Movement

After adjusting the motors' angles, the optimal rotation motion from the simulation was nearly replicated in the real-world, even with some environmental variations. In order to achieve fluid and steady movement, the transition time between frames was slowed down.

6. Results and Discussion

6.1. Model Training

The YOLOv11n (nano) object detection model was trained on the Cube dataset, the object to be manipulated. The coffee bottle and additional foam-made cubes were used to assess the model's performance at first. After evaluating the model's strengths and weaknesses, the actual cube dataset was used to train the model for further hardware implementation.

The prepared cube dataset includes 257 test photos, 602 validation images, and 10,719 training im-

ages of objects—more precisely, cubes and cuboids—that have been manually labelled using assisted annotation techniques. Pre-processing and augmentation steps were applied to the prepared dataset for various scenarios. The model was trained for 110 epochs for optimal object detection with satisfactory confidences for each detection and accurate distance estimation. The trained model was successfully able to detect and recognize green cube, which allows the robot to detect, recognize and reach the identified objects.



Fig. 7. Prediction of Cubical Object

6.2. Object Detection Performance

The trained model's performance was evaluated through multiple real-world object detection tests, as well as metrics that give insights into the performance of the trained object detection model.

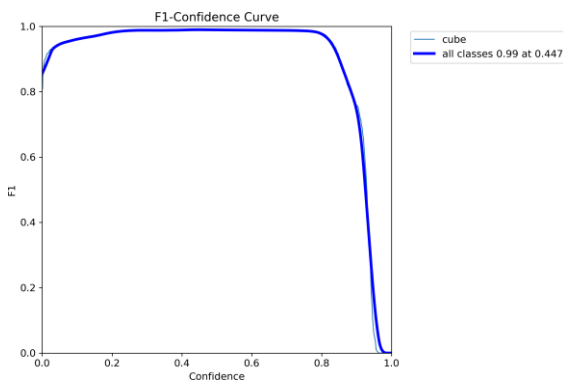


Fig. 8. F1-Curve of trained model for Cube Dataset

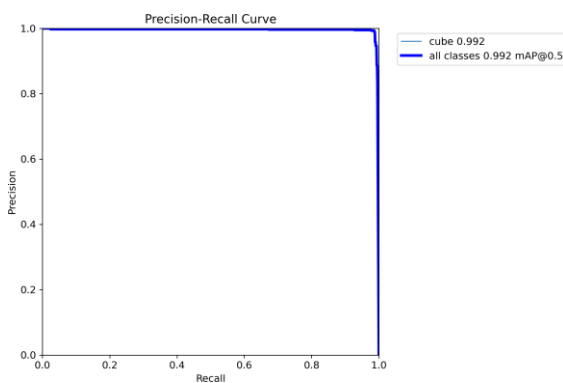


Fig. 9. PR-Curve of trained model for Cube Dataset

The fig.8 shows the F1-confidence curve for cube detection. For each confidence value, corresponding precision-recall pairs are obtained. The F1 score for each precision-recall pair is plotted against the confidence level. The curve approaches a unit value of Area Under Curve (AUC), showing that the model is able to predict the class of objects in the image accurately.

The Precision-Recall curve is shown in the fig.9. For each probability threshold, a pair of precision and recall value is obtained. These data points are then plotted to obtain the PR curve. The area under the PR curve is nearly 1, which shows that the model is able to classify the detected objects in the image accurately.

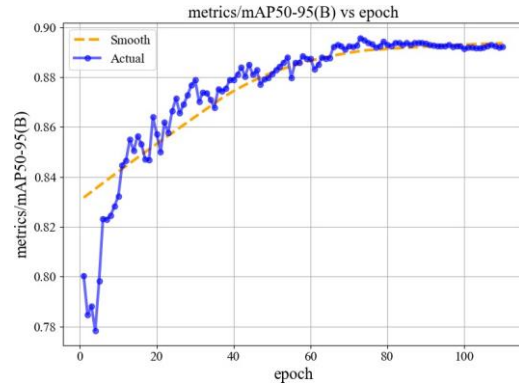


Fig. 10. mAP50-95 of model for Bounding Box

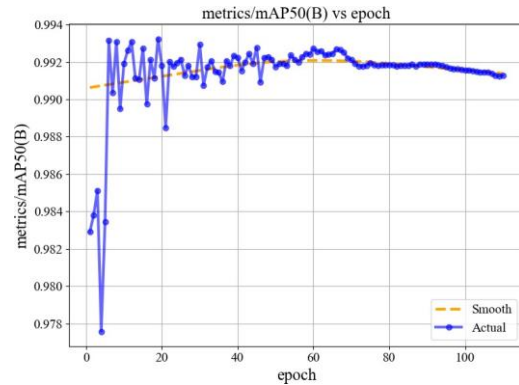


Fig. 11. mAP50 of model for Bounding Box

The curves in fig.10 and fig.11 illustrate the object detection performance of the trained model over 110 training epochs. The curve in fig.10 showing mAP50-95 (B) shows a steady improvement in detection accuracy under changing IoU thresholds (intersection over union), reaching 88.9%, indicating strong bounding box localization and generalization. Similarly, the curve in fig.11 showing mAP50(B), peaks early and remains consistently high around 99.3%, indicating the model's excellent ability to detect and classify objects with relaxed overlap constraints. Together, these curves show the model's ability not only to detect object accurately but also to localize it with high precision.

6.3. Simulation Results

The different sets of movements were seamlessly executed in the simulation environment. All the motions to be executed in the physical robot were tested and simulated in the simulation to get the initial joint data frame by frame. The situation was so ideal that even the fall recovery action was smooth as per the intended programming, along with all other motions such as: Forward, sideways, rotation, and pick-place of

the object. The use of built-in vision sensors of CoppeliaSim was also ideal for detecting and measuring distances towards the object.

The simulation of the robot in the software was highly engine-dependent, causing some unwanted behavior due to a mismatch in real-world conditions and simulated conditions of friction and contact physics.

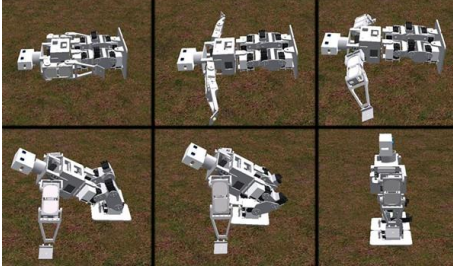


Fig. 12. Fall Recovery in Simulation Facing Up

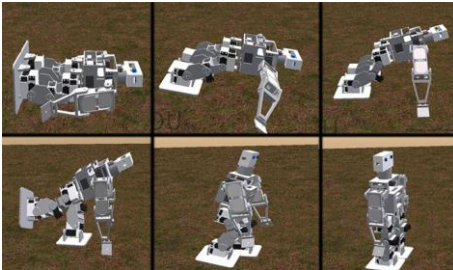


Fig. 13. Fall Recovery in Simulation Facing Down

6.4. Distance Estimation Performance

From multiple experiments with the stereo camera setup, the calculated distance and actual distance were compared. It was found that the calculated distances were most accurate in the range from 10 cm to 60 cm. For instance, when the object was at 29 cm, the calculated distance was 28 cm, but when the object was kept at a distance of 75 cm, it was calculated to be 83 cm.

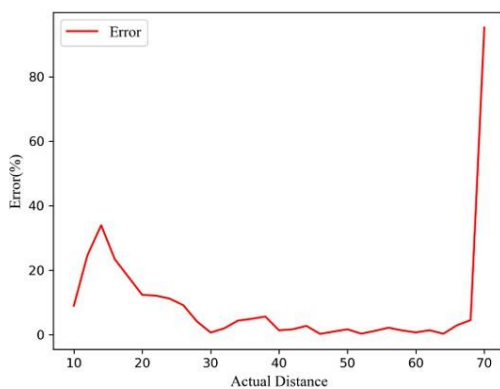


Fig. 14. Error in Distance Calculation

The average accuracy of the distance estimation without considering the last point (last point showed inaccurate result due to a lower number of pixel shift at larger distances) was 93.37%. Taking into account

the final point, 90.52% accuracy was achieved. This demonstrates that the distance estimation performs well for the mentioned range.

The latter was caused by the ESP32-Cam's reduced resolution, which made the bounding box pixel shifts in camera frames when the item was far away gradually indiscernible. As the object was positioned farther away, the object detection confidence score similarly decreased. Therefore, during image processing, items identified with a low confidence score were discarded. As a result, they did not receive an exact distance estimate. The objects were spotted at a greater distance when the image processing step's necessary confidence score was lowered from 80% to 70%.

Also, when the camera's resolution tuned from HVGA to VGA, adding more pixels to the frames, the cube in frames was more precisely covered by the bounding boxes. For objects farther away, this somewhat improved the accuracy of distance calculation.

6.5. Robot Assembly

The finished robot has one plastic gear micro servo (SG90) for the neck and eighteen metal gear servo motors (MG996R) for all of the major joints. The micro-controller and the necessary control circuitry are positioned in the torso, and the head houses two ESP32-Cams with the appropriate dynamics and mass distribution. The finished robot can successfully sense its surroundings utilizing the cameras for stereo vision and execute necessary moves when called upon. It can navigate to the target and operate it using a variety of autonomous maneuvers.

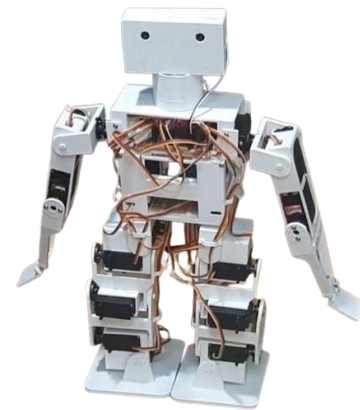


Fig. 15. Assembled Robot

6.6. Motor Initialization

To provide a solid starting pose for the robot, each motor moves sequentially to a predetermined location during initialization. To establish a stable beginning, the procedures were carried out one at a time every second.

The robot is seen in its standing position. This phase guarantees that the subsequent motion or the series of motions to be performed is stable and predictable.

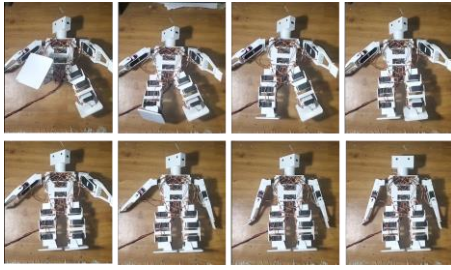


Fig. 16. Motors Initialization

6.7. Walking Sequence

The physics and dynamics of body parts and joints in the simulation software/environment and the real-world differ, so significant adjustments to motion data were required to achieve stable walk motion with stable transitions between frames. The motion of the legs in the physical robot when static was found to be very similar to that of the simulation. The robot moved with a steady posture after the motion data was adjusted, as shown in fig. 17 as well. During a single walking cycle, the robot transfers all of its weight to one leg before moving the other leg a predetermined distance forward. This process is then repeated for the other leg to move an additional step.

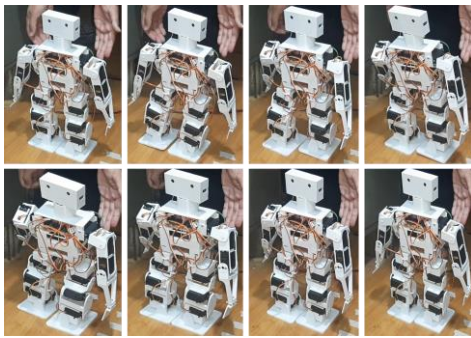


Fig. 17. Forward Walking Sequence

A consistent and seamless transition between cycles is ensured by the introduction of a stabilizing frame in between each cycle. To achieve a greater distance, several walking cycles were done, which allowed the physical robot to properly integrate walking motion. The purpose of this motion is to bring the object and robot closer together.

6.8. Rotation Motion

With a few adjustments to the frame data derived from simulation, the turning of the robot was also successfully executed in the physical robot. The robot turns by correctly shifting its weight on one leg, rotating another leg in a certain direction using the Hip Yaw joint, and then balancing out by lowering all of its legs, which causes the robot to turn in a single direction.

To generate turning motion in both clockwise and counterclockwise directions, the same but mirrored data was used for turning in a different direction. For a greater turning angle, several cycles of the turning motion were repeated. The robot may now turn in the direction of the object thanks to this action.

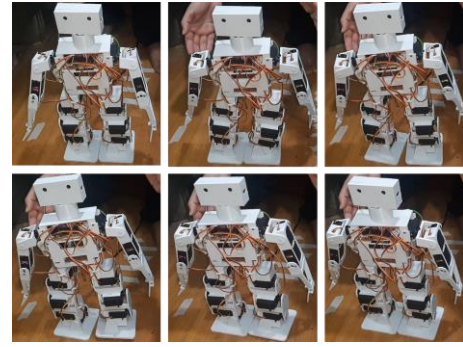


Fig. 18. Rotation Motion

6.9. Sideways Movement

With a slight modification to the motion data derived from simulation, the robot was also able to correctly and successfully execute the sideways movement (left and right). The robot traveled steadily in both left and right directions after modifying sideways movement data from the simulation.

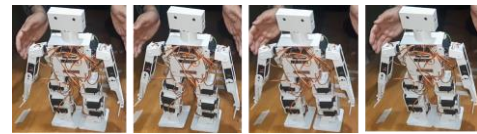


Fig. 19. Sideways Motion

The robot walks sideways by first shifting its entire weight on one leg using Foot Roll and then moving the other leg inward using Hip Roll in either the left or right direction. It then readjusts its posture to return to its starting position and completes one sideways step. To travel farther in any direction, several steps were repeated. Only when the object is not center aligned is this motion carried out, enabling the robot to align itself with the object in the middle of its field of vision for efficient object manipulation.

6.10. Pick/Drop Motion

Pick/Drop motion is the last motion that was effectively incorporated into the actual robot. Although there were some dynamics problems that required some adjustments to the motion data, this motion also functioned utilizing motion data from the simulation. When the pick/drop motion is used in simulation, the robot bends slightly to pick the object; however, when the same motion is used in real life, the robot executes the move successfully but becomes unstable after picking the object.

When the robot arrives at its location, it will pick up and deposit the cube, which is a learned object. This was avoided by removing the robot body's bending, which allowed for stable object pick and drop at every step. With this modification, the robot descends in the same spot as depicted in the figure after picking up the object while standing. When the robot arrives at its destination, this motion enables it to pick and drop the cube, which is a learned object.

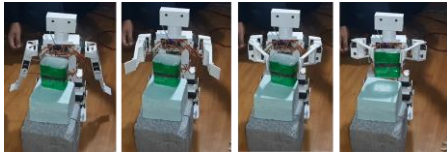


Fig. 20. Pick/Drop Motion

6.11. Motion Behavior Evaluation

The robot's head rotated back and forth to search the surroundings for objects when they were out of its line of sight. After identifying the thing at last, it rotated to face the object. The robot rotated four times with an average rotation angle of 12 degrees when an object was positioned approximately 40 degrees out of its field of vision.

The robot covers almost 30 cm after the execution of 5 walking cycles. The average distance coverage is calculated to be about 5.5 cm per walk cycle. The fluctuation itself rarely goes above 1.5 cm. The net error in distance is also about 2-3 cm, which is not that significant.

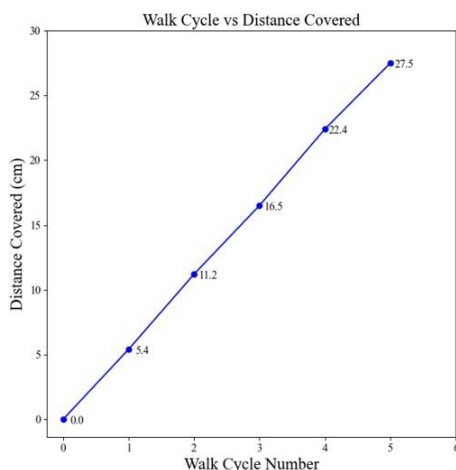


Fig. 21. Walk Cycle vs Distance Covered

6.12. Simulation vs Real World

Some deviations were observed when the simulation model was compared with the real-world implementation. It was found that the data used in the simulation model cannot be directly implemented in real-world motors, due to differences in environmental factors, with the simulation environment being an almost ideal environment, which also depends on the physics engine used. Table. 2 summarizes the comparison between simulation and real-world implementation.

6.13. Technical Challenges

Balance between accuracy and performance: The number of pixel shifts directly affects the object's distance estimation when using the stereo camera setup. Distance estimation becomes inaccurate when the resolution is reduced from SVGA to HVGA because there are fewer pixels overall to represent the images, or

fewer pixel shifts for the object in the stereo arrangement of the camera. It was discovered that precision and CPU overheating must be balanced.

Servo Motor angle multiplier effect: The servo motor can rotate from 0 to 180 degrees. Servo motors positioned in various joints must be turned at a precise angle to move the humanoid robot to the required spot. When the walking cycle was put into practice, it was discovered that each servo motor's rotational angle might fluctuate by ± 2 degrees. Six servos must be turned at a predetermined angle to move a single leg of the robot. The leg's end effector has an inaccuracy of $6 \times 2 = 12$ degrees if all 6 servos cause even a 2-degree error. The robot can be readily dislocated by a 12-degree mistake.

7. Conclusion

Using the prototype hardware, a small-scale humanoid robot is created, and static motions that resemble typical human motions are applied. As a stereo camera configuration, the ESP32 CAMs did rather well, despite being a low-level prototype board. Although the formula and method for estimating distance were likewise precise, as the distance increased, their effectiveness and precision decreased because of the decreasing pixel offset between objects captured by two cameras. However, the usage of servo motors led to several issues with the robot's overall mobility and process motion planning.

Throughout the entire control system, the motions were irregular due to a slight wobble in the servo motor's static position. The rated torque was insufficiently supplied by the MG996R motors. When applied directly to the practical robot, the motion data from the simulation did not function since the observed torque was less than half of the rated torque. The foundation for creating the actual motions, however, was the simulation data, which gave an overview of how the joints should be moved to achieve the intended motion. In order to create motions in these kinds of combinations with several joints, simulation tools might be quite helpful.

8. Future Work

The robot is only dynamic when it is deciding which movements to perform in response to the location of the thing it is trained to grasp. However, all of its movements are determined by frame data, which forces the robot to move in static motion for forward, side, and rotational walking and prevents it from adapting to the surface or environment it is walking on. Therefore, the robot's utility in an industrial setting can be increased by adding sensors such as FSR (Force-Sensing Resistor) sensors for dynamic ZMP (Zero Moment Point) calculations and using reinforcement learning to adapt to changing terrain and environmental circumstances. Although the ESP32 camera works well, object detection and distance calculation can be improved with a higher resolution camera module.

Tab. 2. Simulation vs Real World Comparison

Parameter	Physical	Simulation
Motor Torque	4.8 kg cm	5 kg cm
Fall recovery	Not implemented	implemented
Distance estimation	Below 15-m accuracy was Low	Highly accurate
Per turn rotation	12 degrees	13 degrees
Pick Up motion	Cannot bend to pick the object	Can bend to pick the object
Walking Stability	Transition Frames for stability	No transition frames used
Smoothness	4-ms delay between motions	No delay required
Visual Sensor	Stereo Camera setup	Vision Sensor
Real-time streaming	May lag	No lag

Furthermore, the robot is not able to determine the orientation of the object it is grasping. Therefore, to determine the object's orientation, the YOLO models with OBB (Oriented Bounding Boxes) capabilities can be used. The motions created in this project may be the identical motions needed to align the object in the designated orientation.

AUTHORS

Bal Krishna Shah* – Department of Electronics and Computer Engineering, Thapathali Campus, IOE, TU, Kathmandu, Nepal, ORCID: 0009-0004-7763-8624, e-mail: bal.077bei010@tcioe.edu.np, www: tcioe.edu.np.

Sabin Acharya – Department of Electronics and Computer Engineering, Thapathali Campus, IOE, TU, Kathmandu, Nepal, ORCID: 0009-0006-6614-990X, e-mail: sabin.077bei035@tcioe.edu.np, www: tcioe.edu.np.

Safal Karki – Department of Electronics and Computer Engineering, Thapathali Campus, IOE, TU, Kathmandu, Nepal, ORCID: 0009-0005-5477-88445, e-mail: safal.077bei036@tcioe.edu.np, www: tcioe.edu.np.

Aayush Pathak – Department of Electronics and Computer Engineering, Thapathali Campus, IOE, TU, Kathmandu, Nepal, e-mail: aayush.077bei002@tcioe.edu.np, www: tcioe.edu.np.

Saroj Shakya – Department of Electronics and Computer Engineering, Thapathali Campus, IOE, TU, Kathmandu, Nepal, e-mail: sarojsh@tcioe.edu.np, www: tcioe.edu.np.

*Corresponding author

ACKNOWLEDGEMENTS

We thank Er. Saroj Shakya, Thapathali Campus, and the Department of Electronics and Computer Engineering for their supervision and support.

REFERENCES

- [1] S. Kajita, H. Hirukawa, K. Harada, and K. Yokoi, *Introduction to Humanoid Robotics*, Springer, 2014, <https://doi.org/10.1007/978-3-642-54536-8>.
- [2] C. Jewell, "Bringing ai to life", *World Intellectual Property Organization (WIPO)*, September,

2018, <https://www.wipo.int/web/wipo-magazine/articles/bringing-ai-to-life-40507>.

- [3] G. M. Atmeh, I. Ranatunga, D. O. Popa, K. Subbarao, F. Lewis, and P. Rowe, "Implementation of an adaptive, model-free, learning controller on the atlas robot". In: *2014 American Control Conference (ACC)*, 2014, 2887–2892, doi: 10.1109/ACC.2014.6859431.
- [4] G. Ficht and S. Behnke, "Bipedal humanoid hardware design: a technology review", *Current Robotics Reports*, vol. 2, no. 2, 2021, 201–210, <https://doi.org/10.1007/s43154-021-00050-9>.
- [5] C. G. Atkeson, J. G. Hale, F. Pollick, M. Riley, S. Kotosaka, S. Schaul, T. Shibata, G. Tevatia, A. Ude, S. Vijayakumar, E. Kawato, and M. Kawato, "Using humanoid robots to study human behavior", *IEEE Intelligent Systems and their Applications*, vol. 15, no. 4, 2000, 46–56, doi: 10.1109/5254.867912.
- [6] Y. Duan, X. Chen, R. Houthoof, J. Schulman, and P. Abbeel, "Benchmarking deep reinforcement learning for continuous control", *Computing Research Repository (CoRR)*, vol. abs/1604.06778, 2016, <https://doi.org/10.48550/arXiv.1604.06778>.
- [7] E. Guizzo and E. Ackerman, "The hard lessons of darpa's robotics challenge [news]", *IEEE Spectrum*, vol. 52, no. 8, 2015, 11–13, doi: 10.1109/MSPEC.2015.7164385.
- [8] S. S. Kotha, N. Akter, S. H. Abhi, S. K. Das, M. R. Islam, M. F. Ali, M. H. Ahamed, M. M. Islam, S. K. Sarker, M. F. R. Badal, P. Das, Z. Tasneem, and M. M. Hasan, "Next generation legged robot locomotion: A review on control techniques", *Heliyon*, vol. 10, no. 18, 2024, e37237, <https://doi.org/10.1016/j.heliyon.2024.e37237>.
- [9] L. H. He, Y. Z. Zhou, L. Liu, W. Cao, and J. H. Ma, "Research on object detection and recognition in remote sensing images based on yolov11", *Scientific Reports*, vol. 15, no. 1, 2025, 14032, <https://doi.org/10.1038/s41598-025-96314-x>.
- [10] G. Jocher and J. Qiu. "Ultralytics yolov11, version 11.0.0". GitHub repository: <https://github.com/ultralytics/ultralytics>, 2024. AGPL-3.0 License.

- [11] Z. Li, B.-L. Xu, D. Wu, K. Zhao, S. Chen, M. Lu, and J. Cong, "A yolo-ggcnn based grasping framework for mobile robots in unknown environments", *Expert Systems with Applications*, vol. 225, 2023, 119993, <https://doi.org/10.1016/j.eswa.2023.119993>.
- [12] R. Kumar, S. Lal, S. Kumar, and P. Chand, "Object detection and recognition for a pick and place robot". In: *Asia-Pacific World Congress on Computer Science and Engineering*, 2014, 1–7, doi: 10.1109/APWCCSE.2014.7053853.
- [13] E. Dandil and K. K. Çevik, "Computer vision based distance measurement system using stereo camera view". In: *2019 3rd International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT)*, 2019, 1–4, doi: 10.1109/ISM-SIT.2019.8932817.
- [14] R. H. Abiyev, M. Arslan, I. Gunsul, and A. Cagman, "Robot pathfinding using vision based obstacle detection". In: *2017 3rd IEEE International Conference on Cybernetics (CYBCONF)*, 2017, 1–6, doi: 10.1109/CYBCONF.2017.7985805.
- [15] S. Shamsuddin, L. I. Ismail, H. Yussof, N. I. Zahari, S. Bahari, H. Hashim, and A. Jaffar, "Humanoid robot nao: Review of control and motion exploration". In: *2011 IEEE International Conference on Control System, Computing and Engineering*, 2011, 511–516, doi: 10.1109/ICC-SCE.2011.6190579.
- [16] R. R. Thavai and S. k. N. Kadam, "Inverse kinematics solution for biped robot", *IOSR Journal of Mechanical and Civil Engineering (IOSR-JMCE)*, vol. 12, no. 1, 2015, 57–62, <https://doi.org/10.9790/1684-12145762>.
- [17] M. Ortega-Palacios, A. Palomino-Merino, and F. Reyes-Cortes, "Inverse kinematics model for a 18 degrees of freedom robot", *Journal of Automation, Mobile Robotics and Intelligent Systems*, vol. 17, no. 1, 2023, 22–29, <https://doi.org/10.14313/t4yf9254>.
- [18] D. Huang, W. Fan, Y. Liu, and T. Liu, "Design of a humanoid bipedal robot based on kinematics and dynamics analysis of human lower limbs". In: *Proceedings of the 2020 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, 2020, 759–764, doi: 10.1109/AIM43001.2020.9158973.
- [19] C. S. Pati and R. Kala, "Vision-based robot following using pid control", *Technologies*, vol. 5, no. 2, 2017, 34, <https://doi.org/10.3390/technologies5020034>.
- [20] S. Abdolshah, M. Abdolshah, and S. H. Tang, "Trajectory planning and walking pattern generation of humanoid robot motion", *IAES International Journal of Robotics and Automation (IJRA)*, vol. 4, no. 2, 2014, 135–142, <http://doi.org/10.11591/ijra.v4i2.pp135-142>.
- [21] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "Rma: Rapid motor adaptation for legged robots", *Computing Research Repository (CoRR)*, vol. abs/2107.04034, 2021, <https://doi.org/10.48550/arXiv.2107.04034>.
- [22] R. R. Ojha, R. M. Sharma, S. P. Bhattarai, J. T. Magar, and D. B. Kshatri, "Drink serving robotic arm relying on multimodal inputs", *European Journal of Applied Science, Engineering and Technology*, vol. 3, no. 2, 2025, 261–276, [https://doi.org/10.59324/ejaset.2025.3\(2\).21](https://doi.org/10.59324/ejaset.2025.3(2).21).